

m2cloud — « Hearth »

Plateforme de discussion microservices, *cloud-native* sur AWS EKS

PostgreSQL · Redis · RabbitMQ · Kubernetes · Terraform · GitHub Actions

na2sime — M2 Cloud

Le sujet & les contraintes

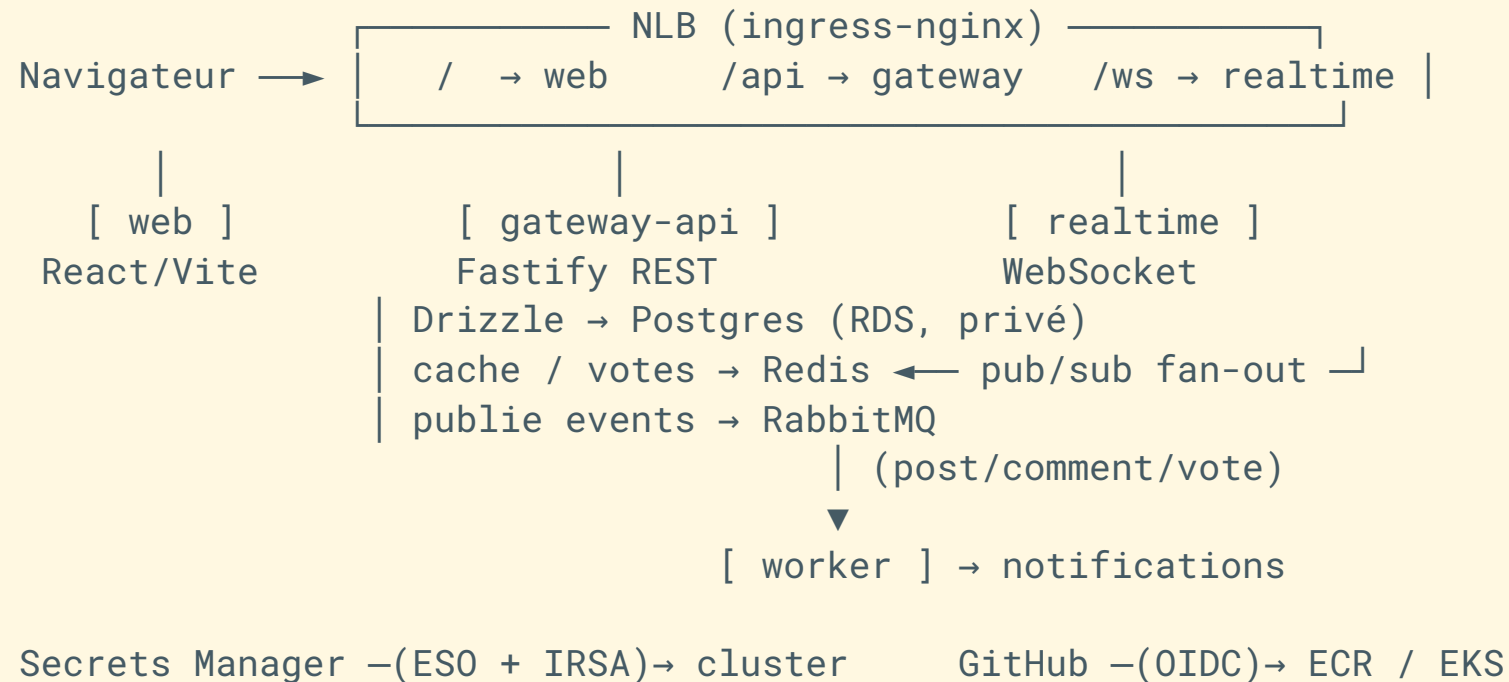
Objectif : une app type Reddit avec **chat temps réel**, en **microservices**, déployée sur une **vraie infra cloud**.

Contraintes imposées :

- PostgreSQL · Redis · RabbitMQ
- **Kubernetes obligatoire**
- Infrastructure-as-Code + CI/CD GitHub
- Monorepo

Le vrai défi : pas une démo locale — du **cloud réel**, déployé, sécurisé, automatisé, en une itération.

Architecture globale



3 backends + 1 front · données relationnelles managées · découplage event-driven

Découpage microservices — *pourquoi 4 ?*

Service	Responsabilité	Profil de scaling
gateway-api	REST, auth JWT, écrit la DB, publie les events	HPA CPU (rafales HTTP)
realtime	WebSocket, fan-out Redis pub/sub	connexions longues
worker	consomme RabbitMQ → notifications	profondeur de queue
web	SPA statique	trafic lecture

Frontières nettes + 3 profils de charge distincts → la vraie raison d'avoir des microservices (pas juste « découper pour découper »).

Ni monolithe (scaling uniforme), ni 6 services (over-engineering pour une soirée).

Choix techniques (1/3) – *Compute*

EKS (Kubernetes) vs Serverless (Lambda/Fargate) vs ECS

Verdict	
Lambda / serverless	✗ WebSocket longues connexions + worker stateful s'orchestrent mal ; cold starts
ECS/Fargate	⚠ ok mais moins de contrôle fin (HPA, probes, PDB)
EKS	✓ scaling fin, portable, standard de marché, démontre la maîtrise infra

“ Imposé *et* justifié : l'app a des besoins (WS, workers, autoscaling par profil) que K8s sert nativement. ”

Choix techniques (2/3) — *Données & ORM*

- **RDS PostgreSQL** vs S3 / DynamoDB → données **fortement relationnelles** (users/posts/comments/votes, unicité, jointures). S3 = objets, DynamoDB = clé-valeur → inadaptés.
- **Drizzle** vs Prisma → ORM **TypeScript-first, léger** (pas de moteur de query séparé), SQL explicite, migrations versionnées, **bundle-friendly** (Prisma embarque un binaire — friction en image légère).
- **Redis & RabbitMQ in-cluster** vs ElastiCache / Amazon MQ → coût ↓ **et** démonstration de la **persistance K8s** (StatefulSet + PVC EBS), item clé du barème.

Choix techniques (3/3) — *Packaging*

Monorepo pnpm : contrats/types partagés (`packages/shared`), CI unifiée, changements atomiques.

Build : tsup bundle + pnpm deploy vs image avec tout le monorepo
→ chaque service = un `dist/index.js` + node_modules de prod **uniquement** → images runtime **minuscules** (~25 Ko de code applicatif).

```
// tsup : bundle les deps workspace, externalise les deps npm  
noExternal: [/^@m2cloud\\//],  
external: ["fastify", "ioredis", "amqplib", "prom-client", ...]
```

Infrastructure as Code — Terraform

100 % Terraform, état distant **S3 + verrou DynamoDB** (reproductible, pas de state local).

Provisionné (88 ressources, 1 `apply`) :

VPC (2 AZ, subnets privés, NAT) → EKS (node group spot, IRSA) → RDS (privé) → ECR
→ Secrets Manager → IAM/OIDC

```
module "eks" {  
  source = "terraform-aws-modules/eks/aws"  
  cluster_version = "1.31"  
  eks_managed_node_groups = { default = { capacity_type = "SPOT", ... } }  
  enable_cluster_creator_admin_permissions = true  
}
```

“ Pourquoi Terraform : multi-cloud, écosystème de modules mûrs, plan/apply explicite. ”

Sécurité & gestion des secrets

Zéro secret dans le code — la pièce maîtresse :

- AWS **Secrets Manager** → **External Secrets Operator** → `Secret` K8s, via **IRSA** (rôle IAM porté par un ServiceAccount, **aucune clé statique**)
- CI/CD : **OIDC GitHub** → la pipeline *assume* un rôle IAM à l'exécution → **aucune access key stockée**
- **IAM least-privilege** : rôle CI = `ECR push` + `eks:Describe` ; rôle ESO = lecture **d'un seul** secret
- **Jamais le compte root** pour les workloads → user IAM dédié `m2cloud-deployer`
- RDS en **subnets privés**, SG verrouillé aux nœuds EKS

Scalabilité & résilience

Axe	Mise en œuvre
Auto-scaling	HPA (CPU 70 %, 2→6) sur gateway & realtime – <i>metrics-server</i>
Stateless	services répliqués, sans état local
Anti-fragilité	<code>topologySpreadConstraints</code> (répartition multi-nœuds), PodDisruptionBudget
Persistence	PVC EBS (Redis/RabbitMQ) + backups RDS + multi-AZ
Santé	probes <code>liveness</code> / <code>readiness</code> / <code>startup</code> , graceful shutdown

“ *Démontrable live* : `kubectl -n app get hpa` → `cpu: 11%/70%` ”

CI/CD — 3 pipelines GitHub Actions

```
ci.yml    PR/push → typecheck · tests (Postgres service) · build
cd.yml    main    → OIDC → build/push ECR → migration Job → rollout EKS
infra.yml PR infra → terraform fmt + validate
```

- **OIDC** plutôt que des clés AWS → surface d'attaque réduite
- **Migration en Job in-cluster** (RDS privé, injoignable de l'extérieur)
- Déploiement **Kustomize** (`kustomize edit set image $sha`)

“ Pipeline complet **vert** : du commit au pod en prod, sans intervention manuelle. ”

Retour d'expérience

Les vraies galères du cloud réel – et comment je les ai résolues

REX (1/2) – Build & provisioning

Symptôme	Cause	Solution
Services crash au boot	esbuild : « <i>Dynamic require not supported</i> » (ioredis/amqplib CJS bundlés en ESM)	externaliser les deps npm, ne bundler que le workspace
Migration → no pg_hba.conf ... no encryption	RDS force le TLS	?sslmode=require dans DATABASE_URL
Pods Pending indéfiniment	EKS 1.31 ne fournit plus de StorageClass par défaut	StorageClass gp3 (EBS CSI) par défaut
RabbitMQ ImagePullBackOff	tag Bitnami retiré du registry public (2025)	image officielle via StatefulSet maison

REX (2/2) — Le bug réseau (le plus intéressant)

Symptôme : `/api` répond (200), mais `/` (web) → **504** dans le navigateur.

Démarche (≠ deviner) :

1. Pods `Ready`, endpoints OK, test interne `web:80` → 200 en 8 ms ✓
2. Logs ingress : `upstream timed out connecting to 10.0.2.x:80`
3. **Matrice de joignabilité** depuis l'ingress : `gw:3000` ✓ · `web:80` ✗ — *sur les deux nœuds*

Cause : le **SG des nœuds EKS n'autorise le cross-node que sur 1025-65535**. Le web écoute sur **80** → bloqué. (Et le smoke ne testait que `/api` → invisible.)

Fix : règle SG port 80 + `topologySpread` + check `/` ajouté au smoke.

REX — ce que j'en retiens

- **Le « ça marche en local » ne suffit pas** : RDS (SSL), EKS (StorageClass, ports SG), registries (Bitnami) ont des comportements propres au cloud.
- **Debugger par isolation** (interne vs externe, matrice de joignabilité) > deviner.
- **Tester ce qu'on déploie vraiment** : un smoke qui ne couvre pas tous les chemins ment.
- **Tout fixer dans l'laC/le repo** → reproductible, pas un patch manuel oublié.

“ Chaque incident est devenu une ligne de code versionnée + un test. ”

Démo

1. `http://...elb.amazonaws.com` → inscription, room, post, vote, **chat live**
2. Le repo : commits, **3 workflows verts**, `infra/`
3. `kubectl -n app get pods,hpa,pvc` → replicas, autoscaling, persistance

“ *App réellement déployée sur AWS EKS — full e2e validé (REST + notif async RabbitMQ + WebSocket).* ”

Bilan & pistes d'amélioration

Acquis : archi microservices justifiée · IaC complète · sécu (OIDC/IRSA/ESO, no-root) · CI/CD vert · déployé & opéré sur du vrai cloud.

Pour aller plus loin :

- **AWS Load Balancer Controller** (NLB IP-target) → supprime le souci de health-check / cross-node
- **KEDA** → autoscaling du worker sur la profondeur de queue RabbitMQ
- **Prometheus + Grafana** (endpoints `/metrics` déjà prêts)
- **GitOps (ArgoCD) + Karpenter** (autoscaling de nœuds)

Merci !

Questions ?

`github.com/na2sime/m2cloud`